

FREQUENCY (GENERAL / NUMERIC)

```
int freq[100005] = {0}; freq[x]++; // bounded values, fast array
map<int,int> freq; freq[x]++; // sparse/large values
unordered_map<int,int> freq; freq[x]++; // faster, unordered
int mx = 0, val = -1; for (auto &[k,v] : freq) if (v > mx) { mx = v; val = k; }
// most frequent
if (freq.count(x)) { /* x exists */ } // check existence in map
if (freq.count(target-x)) { /* pair found */ } // count pairs with sum = target
set<int> s(v.begin(), v.end()); s.size(); // count of distinct elements
// two-sum, return indices
map<int,int> seen; for(int i=0;i<n;i++){ if(seen.count(t-a[i])) {/*pair: seen[t-a[i]],i*/} seen[a[i]]=i; }
```

SUBARRAY SUM = K (PREFIX SUM + FREQ MAP)

```
map<long long,int> cntPre; cntPre[0] = 1; long long pre = 0, ans = 0;
for (int x : a) { pre += x; ans += cntPre[pre - k]; cntPre[pre]++; } // count
subarrays summing to k
```

SORTING – CUSTOM COMPARATORS

```
sort(vp.begin(), vp.end(), [](pair<int,int> a, pair<int,int> b){ return a.second <
b.second; }); // sort pairs by 2nd
sort(words.begin(), words.end(), [](string a, string b){ return a.size() < b.size();
}); // sort by length
stable_sort(v.begin(), v.end()); // keeps equal order partial_sort(v.begin(),
v.begin()+k, v.end()); // sorts first k only
sort(v.rbegin(), v.rend()); // descending, alt to greater<int>()
sort(vp.begin(), vp.end(), [](pair<int,int> a, pair<int,int> b){ if(a.first!=b.first)
return a.first<b.first; return a.second>b.second; }); //tie-break: 1st asc, 2nd desc
```

BASIC MATH

```
abs(x); // absolute value
max({a,b,c}); min({a,b,c}); // multi-arg max/min
((a % m) + m) % m; // fix negative modulo
(a + b - 1) / b; // ceil division a/b
n*(n+1)/2; // sum 1..n n*(n+1)*(2*n+1)/6; // sum of squares 1..n
(x2-x1)*(x2-x1) + (y2-y1)*(y2-y1); // squared distance (avoid sqrt/float)
```

```
int mx=INT_MIN, mx2=INT_MIN; for(int x:a){ if(x>mx){mx2=mx;mx=x;}
else if(x>mx2 && x!=mx) mx2=x; } // 2nd largest, single pass
```

```
long long res=1; for(int i=0;i<b;i++) res*=a; // a^b, manual (avoid pow() float)
```

```
is_sorted(v.begin(), v.end()); // true if already sorted
```

ARITHMETIC PROGRESSION SUM

```
Sn = n/2 * (a + l);
Sn = n/2 * (2*a + (n-1)*d);
// Sn = sum of first n terms
// a = first term
// l = last term
// d = common difference
// n = number of terms
```